

All students should watch the videos and review the readings accompanying this LGA in preparation for this assignment and the next lecture.

Individual self-test questions

All students should answer this set of questions. You will compare your answers with your group during the next lecture period. Note, we may from time to time include trick questions.

- “FIFO” stands for _____ and describes the _____ ADT.
- “LIFO” stands for _____ and describes the _____ ADT.
- The three principle operations on a stack are _____, _____, and _____. These operations allow interaction only with the _____ of the stack.
- The three principle operations on a queue are _____, _____, and _____. Which end of the queue does each operation affect?

Group questions

Distribute the following questions across the members of your group. You will share your solutions (and most importantly the *method* of your solutions) during the next lecture period. Divide up the questions so that each question has at least three solutions from different group members.

1. (Basic) Write a C++ function that, given a stack of integers, swaps the top two elements in the stack; if the stack contains fewer than two elements, the function should do nothing.

2. (Intermediate) You are given a large box. Inside the box is another, smaller box; inside that box is a yet smaller box; and so forth. There are n boxes in total. Each box is labeled with a unique label.

Your job is to separate out all the boxes. Another worker is waiting to take each box as soon as you open it and remove its contents; you only have time enough to quickly record the label on the box before it is taken away. You are only able to use a stack data structure for recording the labels. Your co-worker moves all of the boxes to a new location.

The next day, your job is to put all of the boxes back as they were - smaller into larger. However, your boss is a real stickler for efficiency, and will only let you place a smaller box into a larger box a total of $n - 1$ times - which means you cannot make any mistakes. Unfortunately, many of the boxes are similar in size, there are a lot of boxes, and your careless co-worker scattered the boxes around any which way. Good thing you have your stack!

Describe your algorithm for repacking all of the boxes.

3. (Intermediate) Same problem, but you are given a queue instead of a stack.

4. (Intermediate) A palindrome is a string that reads the same forward and backward, such as “kayak”, “civic”, or “racecar”. Describe one way you can use a stack and a queue together to determine if a given string is a palindrome.

5. (Basic) What is the output of the following code? (Drawing the stack and queue may be helpful!)

```
stack<string> s;
queue<string> q;

s.push("green");
q.push("green");
s.push("grey");
q.push("grey");

s.pop();
q.pop();

cout << s.top() << endl;
cout << q.front() << endl;

s.push("yellow");
q.push("yellow");
s.push("purple");
q.push("purple");

s.pop();
q.pop();

cout << s.top() << endl;
cout << q.front() << endl;

s.push("orange");
q.push("orange");

s.pop();
q.pop();
s.pop();
q.pop();

cout << s.top() << endl;
cout << q.front() << endl;
```

For Further Exploration

Reverse Polish Notation (RPN) or postfix is an arithmetic expression notation where operators follow operands. For example: “3 + 4” is written as “3 4 +” in RPN. RPN notation may be used to write composite expressions such as “5 8 6 - ×”, which would be “5 × (8 - 6)” in infix notation.

Evaluation of expressions is accomplished using a stack. The expression is read left-to-right. As operands are encountered, they are pushed onto the stack. When an operator is encountered, the necessary number of operands are popped from the stack, the operator is applied, and the result is pushed back onto the stack.

Let’s trace through the example above, “5 8 6 - ×”: first, we push 5, then 8, and then 6 onto the stack. The first operator encountered is the minus operator; since 8 and 6 are the top two elements of the stack at that point, those operands are removed from the stack, and the difference (2) is pushed back onto the stack. This leaves the 5 and the 2 on the stack for the final multiplication operation.

One advantage of the RPN is that you never need parentheses to indicate the order of expression evaluation! HP’s line of pocket calculators are famous for their use of RPN and stack evaluation. Read more here: <https://www.hpmuseum.org/rpn.htm>.

Using the STL stack, can you write a simple program to evaluate arithmetic expressions in RPN?